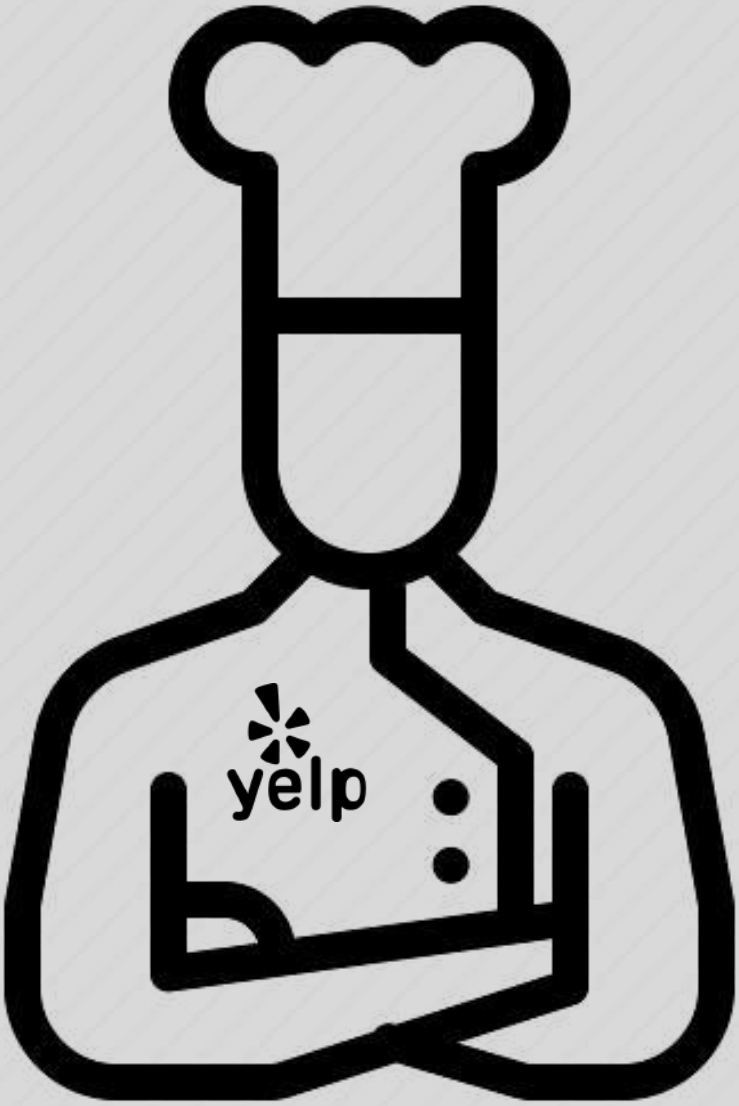
I.A. RESTAURANTERA

DIPLOMADO EN CIENCIA DE DATOS



FERNANDO BARRANCO RODRIGUEZ

INTELIGENCIA ARTIFICIAL APLICADA AL SECTOR RESTAURANTERO

Índice

1. Introducción **2**

2. Planteamiento del problema **3**

3. Descripción de los datos **4**

4. Ingeniería de datos **5**
4.1. Preprocesamiento de texto 5
4.1.1. Remoción de palabras vacías 6
4.1.2. Lematización 6
4.1.3. Detección de frases 7
4.2. Vectorización de palabras 8
4.2.1. CBOW 9
4.2.2. Skip-gram 9

5. Análisis exploratorio **11**
5.1. Univariado 11
5.1.1. Ley de Zipf 11
5.1.2. Etiquetado gramatical 12
5.1.3. Reconocimiento de entidades nombradas 13
5.2. Bivariado 14
5.2.1. Similitud de términos 14
5.2.2. Álgebra de palabras 15
5.3. Multivariado 15
5.3.1. t-SNE 15

6. Modelación no supervisada **17**
6.1. Modelado de tópicos 17
6.1.1. Asignación Latente de Dirichlet 17
6.2. Remoción de ruido en imágenes 21
6.2.1. Autoencoders 22

7. Modelación supervisada **23**
7.1. Análisis de sentimientos 23
7.1.1. Categorización de texto 23

8. Estrategia de negocio **25**
8.1. Sistema de recomendación dual 25

9. Conclusiones **28**
9.1. Siguientes pasos 29

1. Introducción

Este documento analiza la información en forma de texto e imágenes de la empresa pública Yelp con la finalidad de generar valor a través del estudio de los datos que año con año libera la compañía sin fines de lucro. Yelp es una empresa pública estadounidense con sede en San Francisco, California. La compañía desarrolla, aloja y comercializa el sitio web de *www.yelp.com* y la aplicación móvil de Yelp donde publican reseñas de empresas de fuentes públicas. También opera un servicio de reservas en línea llamado Reservas de Yelp.

Las razones de estudio son esencialmente promovidas para ayudar a impulsar la toma de decisiones basadas en un criterio analítico y fundamentadas en los datos, haciendo uso del método científico para lograrlo. El tema de investigación es clave para fortalecer un ecosistema hoy en día aislado como es el sector restaurantero en términos de análisis de datos, siendo el principal objetivo ampliar el campo de la Inteligencia Artificial, para auxiliar y no reemplazar, a los profesionales con experiencia limitada en Ciencia de Datos, permitiéndoles mejorar, agilizar, y ampliar el entendimiento de su negocio y los procesos que ocurren dentro de él.

2. Planteamiento del problema

El ecosistema restaurantero ha tenido un progreso mínimo en la toma de decisiones basadas en datos en comparación con los sectores que sí han aprovechado esta oportunidad y se han desarrollado de forma acelerada como son áreas de tecnología, finanzas, transporte, medicina, etc. La evolución de estos se ha dado principalmente por la inversión en investigación de nuevas tecnologías y algoritmos que tengan la capacidad de hacer frente a sus competidores.

Los objetivos que se plantean en este análisis buscan poner un freno de mano al estancamiento analítico en este sector, dando una solución bajo los siguientes puntos:

- Generar un sistema de recomendación que permita conectar proveedores y consumidores dentro del ramo restaurantero basado en los perfiles de los clientes y que se menciona de los establecimientos, tomando como punto de partida los comentarios redactados por los usuarios de la plataforma de Yelp, y con el apoyo de técnicas de aprendizaje máquina y procesamiento del lenguaje natural sugerir el restaurante "ideal" a cada cliente con base en sus preferencias; brindando a su vez a cada restaurante una calificación para cada uno de los rubros resultantes de analizar las millones de críticas.
- Mejorar la calidad de las fotografías que suben los consumidores a la aplicación con la finalidad de que otros clientes puedan ver estas imágenes con un mayor grado de nitidez y así mejorar la experiencia de usuario con el apoyo de técnicas de aprendizaje profundo y visión artificial.

Actualmente el sector no cuenta con los puntos previamente mencionados, es por ello que se propone una estrategia de negocio similar al de empresas como Netflix, Spotify o Youtube (Google), basado en un sistema de recomendación con la finalidad de aumentar el número de clientes en la plataforma buscando entonces mejorar la experiencia de usuario brindándole la facilidad de encontrar lo que está buscando y así ampliar la red de usuarios dispuestos a interactuar con la aplicación consiguiendo un incremento en las ganancias.

3. Descripción de los datos

Los datos a trabajar a lo largo del proyecto son un subconjunto de datos públicos en formato *json* de la empresa Yelp que libera año con año con propósitos educativos y académicos como ellos mismos mencionan en su página web: `https://www.yelp.com/dataset`.

El conjunto de datos disponible cuenta actualmente con 8,021,122 de comentarios que realizan los usuarios sobre más de 209,393 negocios dentro de 10 áreas metropolitanas en los Estados Unidos de América. Cuenta también con 200,000 fotografías que los clientes toman y suben desde la aplicación móvil de Yelp.

Adicionalmente el dataset cuenta con 1,320,761 recomendaciones de 1,968,703 de usuarios y más de 1.4 millones de atributos comerciales. El peso aproximado de los archivos en formato *json* se encuentra alrededor de los 9.8GB de espacio en disco, y las fotografías poco más de 7.2GB.

En caso de no ser suficientes registros, Yelp cuenta también con su propia API para desarrolladores interesados en obtener datos de los más de 50 millones de negocios ubicados alrededor del mundo en la siguiente liga: `https://www.yelp.com/fusion`

El dataset cuenta con 1,336 categorías distintas entre las que se encuentran los **Restaurantes**, donde tras una pequeña limpieza se obtienen 63,944 de ellos y texto de 5,058,162 comentarios que forma el corpus[1] de trabajo.

[1]Un corpus es una gran colección de texto escrito en el que se basa un análisis lingüístico.

4. Ingeniería de datos

Trabajar con una cantidad de datos extensa requiere del apoyo de las herramientas adecuadas para llevar a cabo tareas de limpieza, tratamiento e ingeniería de datos, es por ello que el uso de un lenguaje de programación es vital para el manejo de todas estas tareas. De aquí en adelante debe ser sabido que el uso de *Python* y algunas de sus librerías son utilizadas para la manipulación y modelado de datos. Entre las librerías y frameworks a destacar por el protagonismo e importancia se resalta el uso de *spacy*, *scikit-learn*, *gensim*, *pandas* y *numpy* entre otras.

4.1. Preprocesamiento de texto

Preprocesar el texto significa llevarlo a una forma que sea predecible y analizable para realizar determinada tarea. El preprocesamiento ideal de una tarea puede convertirse en algo perjudicial para otra. El procesamiento previo de texto no es directamente transferible de una tarea a otra.

Por ejemplo, al tratar de descubrir palabras de uso común en un conjunto de datos, si algún paso de preprocesamiento implica eliminar *stopwords* porque alguna otra tarea lo usó, entonces probablemente se perderán algunas de las palabras comunes, debido a que ya se han eliminado. Por lo tanto es un enfoque único para cada caso.

Entre los tratamientos menores de limpieza de datos que se realizan, se encuentran los siguientes:

- Remoción de acentos
- Remoción de signos de puntación
- Remoción de símbolos
- Remoción de caracteres especiales
- Remoción de *hapaxes*[2]

Se aplican también otros tratamientos más robustos los cuales se profundizan en los siguientes apartados, pues la complejidad de realizar estas tareas merece un mayor nivel de entendimiento y estudio.

Se presenta a continuación un ejemplo de los millones de comentarios escritos por los usuarios de la aplicación que será utilizado a lo largo del documento, **debe asumirse**

[2]Un hápax o ápax es una palabra que ha aparecido registrada solamente una vez en un corpus.

que cualquier tipo de limpieza, transformación o modelación aplicada a este texto o a cualquier otro ejemplo que se provea aplica sin pérdida de generalidad a el corpus completo.

> *"Visited on Mon May 19, was surprised to find this place open on Victoria Day. A handful of people were there, we sat on the patio. We ordered the double-dipped wings, which are wings which are baked in one flavour, and then fried with a second flavour. I have never tried this before, and this should be a huge reason to come to this place! We had two pounds one was The Angry Mexican (Tequila Lime then Hot) and The Hoser Eh (Maple and Rye Spiked BBQ Sauce, dusting of double smoked candied bacon).*
> *The server said The Hoser Eh recently won some wing competition award. The service was excellent, food was excellent. That afternoon there were 3 patrons on the patio that were arguing loudly for over 20min, they almost started to fight. Aside from that, would definitely come back again. There is pay and display parking along Avenue Road."*

4.1.1. Remoción de palabras vacías

Palabras vacías o *stopwords* es el nombre que reciben las palabras sin significado como artículos, pronombres, preposiciones, etc que son filtradas antes o después del procesamiento de datos en lenguaje natural. No hay una lista definitiva de palabras vacías que todas las herramientas de procesamiento de lenguajes naturales incorporen y no todas usan una lista de palabras vacías. Algunas aplicaciones evitan usarlo específicamente para soportar búsquedas por frase, es por ello que para este análisis no se remueven hasta un paso previo a la modelación, adicionalmente el uso de la lematización puede reducir parte de la base lógica o dependencia de una lista de palabras vacías a filtrar. Por ello se debe tener precaución al momento de realizar una remoción de *stopwords* pues puede llegar a ser una operación que más allá de beneficiar al análisis resulte contraproducente para el mismo.

4.1.2. Lematización

La lematización es muy similar al *stemming*[3], donde el objetivo es eliminar las inflexiones y asignar una palabra a su forma raíz. La única diferencia es que la lematización trata de hacerlo de manera adecuada. No solo corta las palabras, sino que intenta transformarlas a la raíz real. Por ejemplo, la palabra *dijéramos* se transformaría a *decir* y no a *dije*.

Para nuestro caso de uso se muestra a continuación una visualización de la lematización llevada a cabo con el apoyo de la librería *spacy*, donde en caso de que la palabra sea un pronombre o puntuación no cambia a su forma raíz pues esta no debería ser alterada. Nótese que la lematización se realiza de manera adecuada para el idioma inglés. Adicionalmente se muestra la forma raíz de la palabra, ya sea que esté escrita en mayúsculas,

[3]Stemming es un método para reducir una palabra a su raíz o *stem*, el algoritmo más común para stemming es el algoritmo de Porter.

minúsculas, solo la primera en mayúsculas, la longitud de la palabra o si es signo de puntuación.

Token	Lema	Palabra vacía	Espaciado	Forma
Visited	visit			Xxxxx
on	on	Yes		xx
Mon	Mon			Xxx
May	May	Yes		Xxx
19	19			dd
,	,			,
was	be	Yes		xxx
surprised	surprised			xxxx
⋮	⋮	⋮	⋮	⋮
pay	pay			xxx
and	and	Yes		xxx
display	display			xxxx
parking	parking			xxxx
along	along	Yes		xxxx
Avenue	Avenue			Xxxxx
Road	Road			Xxxx

4.1.3. Detección de frases

Similar a los *n-gramos* pero en lugar de obtener subsecuencias de n elementos por secuencia y generando una matriz dispersa, detecta frases de uso frecuente y las une.

Introducido también por Mikolov en 2013, se propone aprender la representación vectorial de frases, que tienen un significado que no es una simple composición de los significados de sus palabras individuales. De esta manera, podemos formar muchas frases razonables sin aumentar mucho el tamaño del vocabulario. La siguiente fórmula expresa el modelado de frases en pocas palabras:

$$\frac{count(AB) - count_{min}}{count(A) \times count(B)} \times N > threshold$$

Esto se puede calcular fácilmente con ayuda de la libraría *gensim* teniendo en cuenta los siguientes parámetros:

- $count(A)$ es el número de veces que el token[4] A aparece en el corpus

[4] *token* es otra forma de llamarle a una palabra en el campo del procesamiento del lenguaje natural (NLP).

- $count(B)$ es el número de veces que el token B aparece en el corpus
- $count(AB)$ es el número de veces que los token AB aparecen en el corpus en ese orden
- N es el tamaño total del vocabulario del corpus
- $count_{min}$ es un parámetro definido por el usuario para garantizar que las frases aceptadas ocurran un número mínimo de veces, el valor predeterminado en la función *Phrases()* de *gensim* es 5
- $threshold$ es un parámetro definido por el usuario para controlar qué tan fuerte es la relación entre dos tokens que requiere el modelo antes de aceptarlos como una frase, el umbral predeterminado utilizado en la función *Phrases()* de *gensim* es 10

Original:

> *"Food was piping hot, which is great!! Had the buffet...and it truly is all you can eat!! You order whatever you want off the menu (except chitlins) and they bring it out to you!!! Mac n cheese, greens, and black eyed peas were pretty good!! Fried chicken was pretty good too! Will come back!!"*

Transformado en Bigramas:

> *"food pipe_hot great buffet truly eat order want menu chitlin bring mac_n cheese green black_eyed pea pretty good fry chicken pretty good come"*

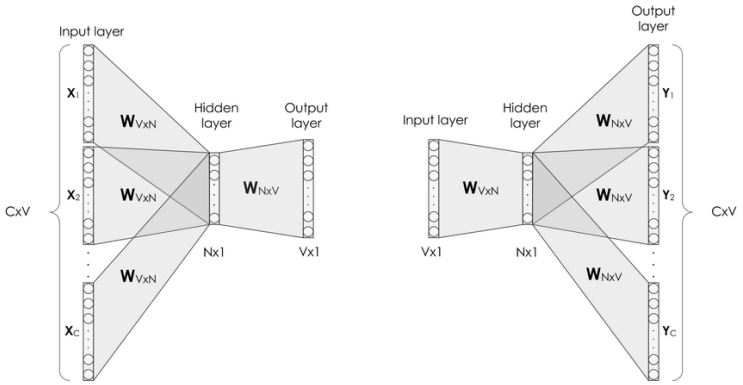
Transformado en Trigramas:

> *"food pipe_hot great buffet truly eat order want menu chitlin bring mac_n_cheese green black_eyed_pea pretty good fry chicken pretty good come"*

4.2. Vectorización de palabras

La incrustación de palabras es una de las representaciones más utilizadas dentro del campo del procesamiento del lenguaje natural (NLP) pues es capaz de capturar el contexto de una palabra en un documento, su similitud semántica, sintáctica y la relación con otras palabras a diferencia de otras técnicas menos poderosas como la vectorización por conteo o el tf-idf. Hablando en términos generales, son representaciones vectoriales de una palabra en particular.

Word2Vec es un método para construir incrustaciones. Se puede obtener utilizando dos métodos (ambos involucran redes neuronales): *skip-gram* y *continuous bag of words*(CBOW).



A la izquierda de la imagen se muestra la arquitectura CBOW y a la derecha la arquitectura Skip-gram.

4.2.1. CBOW

Este método toma el contexto de cada palabra como entrada y trata de predecir la palabra correspondiente al contexto, maximizando la probabilidad de la palabra objetivo mirando el contexto. Y esto resulta ser un problema para las palabras raras. Por ejemplo, dado el contexto, *Ayer fue un día muy ...*, el modelo CBOW dirá que lo más probable es que la palabra sea *bello* o *hermoso*. Palabras como *encantador* recibirán mucha menos atención del modelo, porque está diseñado para predecir la palabra más probable. Esta palabra se suavizará en muchos ejemplos con palabras más frecuentes.

4.2.2. Skip-gram

Por otro lado, el modelo skip-gram está diseñado para predecir el contexto. Dada la palabra *encantador*, debe entenderla y decirnos que existe una gran probabilidad de que el contexto sea *Ayer fué un día muy ...*, o algún otro contexto relevante. Con skip-gram, la palabra *encantador* no intentará competir con la palabra *bello* o *hermoso* sino que los pares (*encantador, contexto*) se tratarán como nuevas observaciones.

De acuerdo con Tomas Mikolov[5] en su publicación *"Distributed Representations of Words and Phrases and their Compositionality"*:

[5]Creador del algoritmo *word2vec*, lo hizo público por primera vez en 2013 mientras trabajaba en Google.

- Skip-gram funciona bien con una pequeña cantidad de datos de entrenamiento, representa bien incluso palabras o frases raras.

- CBOW es varias veces más rápido para entrenar que el salto de gramo, una precisión ligeramente mejor para las palabras frecuentes.

Lo mencionado anteriormente nos da entonces la justificación teórica para utilizar el algoritmo Word2vec en su variante CBOW para modelar el fenómeno en cuestión.

Token	$\cdots$	**94**	**95**	**96**	**97**	**98**	**99**
holiday		-0.047615	0.150039	-0.018545	0.037166	0.146312	-0.033694
san_diego		0.148574	0.031524	-0.085909	0.151252	0.110774	0.028780
definately		0.025630	0.063937	-0.011916	0.181783	0.007077	0.004845
typically		0.113771	0.198106	0.006457	0.001891	0.025035	0.052678
$\vdots$	$\cdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true		-0.023727	0.074684	0.044294	0.198562	0.141496	-0.248111
seat		0.074001	0.132905	-0.137128	-0.100353	0.076991	-0.058579
realize		0.205811	0.112766	-0.018472	-0.163403	0.191467	-0.028814
trick		0.048734	0.009558	0.107020	-0.110675	0.181547	-0.053171

El resultado final es la representación vectorial de cada uno de los $54,583$ términos del vocabulario que compone el corpus en un hiper-espacio de 100 dimensiones. Nótese que la tabla mostrada representa una muestra, el tamaño original es de $54,583$ por 100 donde el índice va de 0 a 99.

5. Análisis exploratorio

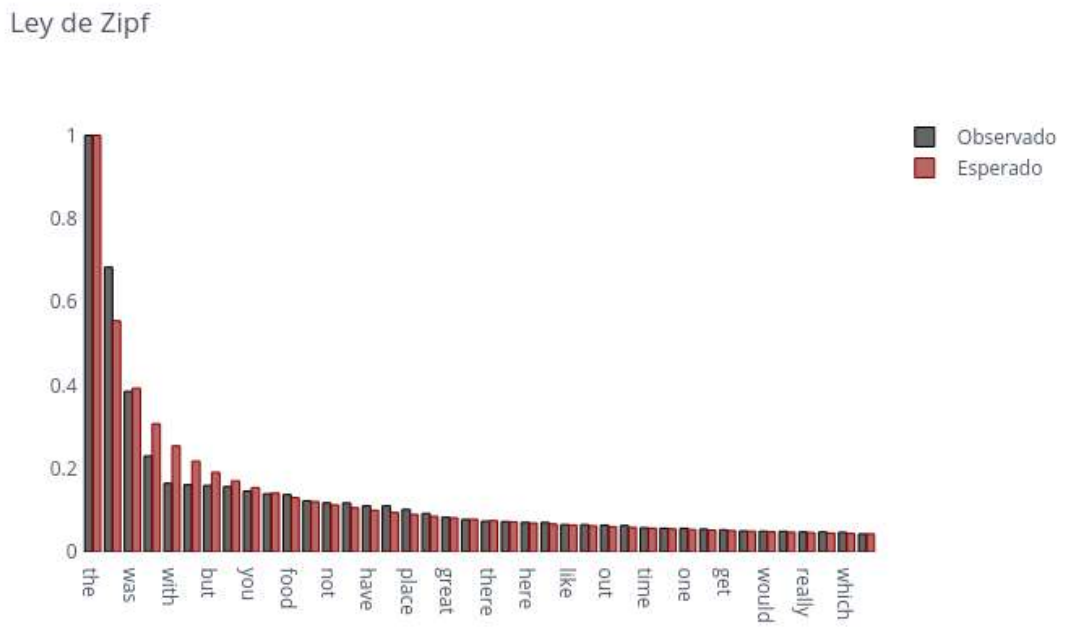
El análisis exploratorio de datos o EDA tiene como objetivo extraer la mayor cantidad de información, conocimiento y patrones del objeto de estudio a través de análisis estadísticos para próximamente modelar el fenómeno en cuestión. Se toman tres enfoques con el fin de "torturar" a los datos y poderlos interpretar sin realizar ningún supuesto sobre ellos, análisis exploratorio univariado (variable por variable), bivariado (pares de variables) y multivariado (múltiples variables).

5.1. Univariado

Como su nombre lo indica, se analiza una a una las variables del objeto de estudio, en este caso las palabras de los comentarios de todo el corpus tanto previa como posteriormente a aplicar las transformaciones descritas en el Capítulo 4.

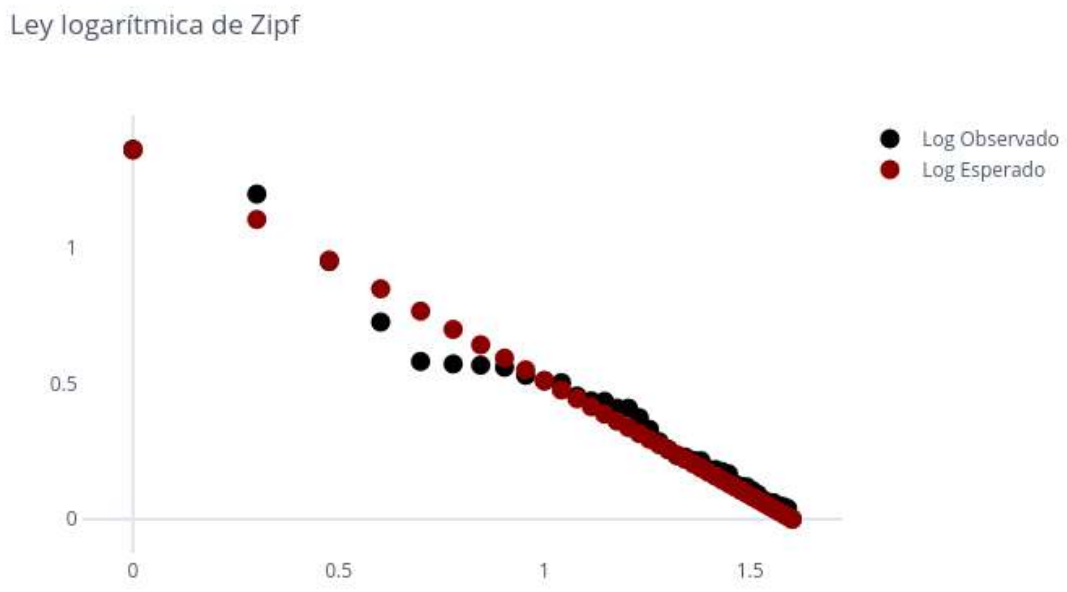
5.1.1. Ley de Zipf

La Ley de Zipf establece que se usa una pequeña cantidad de palabras todo el tiempo, mientras que la gran mayoría se usa muy raramente. Dados algunos corpus de expresiones del lenguaje natural, la frecuencia de cualquier palabra es inversamente proporcional a su rango en la tabla de frecuencias. Así, la palabra más frecuente aparecerá aproximadamente el doble de veces que la segunda palabra más frecuente, tres veces más que la tercera palabra más frecuente, etc.



A pesar de que se puede observar que se sigue la tendencia de la Ley de Zipf, parece que tiene menos área por debajo de la curva Zipf esperada en palabras de mayor rango.

Otra forma de trazar esto es en un gráfico *log* vs *log*, donde el eje X es $\log(rango)$, el eje Y es $\log(frecuencia)$. Al trazar en una escala logarítmica el resultado arrojará una línea aproximadamente lineal en el gráfico.



Si bien esta "ley" está justificada estadísticamente hablando, no se puede concluir que el no cumplirla resulte en algo perjudicial en el análisis por cumplir o no cumplir los supuestos, es considerada solamente una herramienta de visualización (como muchas otras) dado que es interesante ver que el patrón se cumple con mayor precisión entre mayor sea el volumen de los datos.

5.1.2. Etiquetado gramatical

En lingüística computacional el etiquetado gramatical, conocido también por su nombre en inglés *part-of-speech tagging, POS tagging* o *POST*, es el proceso de marcar o etiquetar una palabra en un texto o *corpus*[6] a cada una de las palabras con su categoría gramatical basado tanto en su definición como en su contexto, por ejemplo para la identificación de sustantivos, verbos, adjetivos, adverbios, etc.

[6]Un corpus lingüístico es un conjunto de textos relativamente grande creado independientemente de sus posibles formas o usos, estructurado de ejemplos reales de uso de la lengua.

5.1.3. Reconocimiento de entidades nombradas

El Reconocimiento de entidades nombradas o Name Entity Recognition (*NER*) por sus siglas en inglés, también conocido como extracción de entidades es una tarea que busca localizar y clasificar en categorías predefinidas, como personas, organizaciones, lugares, expresiones de tiempo y cantidades, entidades o palabras sobre bloques de texto no anotado.

Token	**Entidad**	**Puntuación**	**Número**	**Etiqueta gramatical**
Visited				VERB
on				ADP
Mon	DATE			PROPN
May	DATE			PROPN
19	DATE		Yes	NUM
,		Yes		PUNCT
was				AUX
surprised				ADJ
⋮	⋮	⋮	⋮	⋮
pay				NOUN
and				CCONJ
display				VERB
parking				NOUN
along				ADP
Avenue	FAC			PROPN
Road	FAC			PROPN

Los tipos de entidades y etiquetado gramatical así como sus significados se pueden encontrar en la documentación oficial de *spacy* en el siguiente enlace: `https://spacy.io/api/annotation#named-entities`

5.2. Bivariado

El análisis opera sobre aquellos sucesos en los que intervienen dos variables simultáneamente.

5.2.1. Similitud de términos

Word2vec es simplemente una representación vectorial de palabras o *word embeddings* en n dimensiones del espacio, en nuestro caso 100. Se utiliza la similitud de coseno para obtener "semejanza" entre dos palabras, donde dados dos vectores A y B la similitud del coseno se representa como:

$$\text{similitud}(A, B) = \cos(\theta) = \frac{A \cdot B}{||A|| \cdot ||B||}$$

La distancia cosenoidal no es más que obtener la distancia entre dos vectores en un espacio de dimensión n. La distancia representa cómo se relacionan las palabras entre sí.

Ejemplo 1: "pasta"

Token	Similitud
rigatoni	0.854
penne	0.849
fettuccine	0.847
fettuccini	0.845
spaghetti	0.842
manicotti	0.829
linguini	0.827

Ejemplo 2: "tacos"

Token	Similitud
taco	0.884
tostadas	0.838
taquitos	0.827
quesadillas	0.807
pastor	0.788
carne_asada	0.771
tinga	0.769